

program.asm:

```
STRING Enter_a_number:_  
STRING The_double_is:_  
STRING ~
```

```
DATA n 0
```

```
LABEL start  
OUTSN 0  
READN R1 0  
JZER R1 end  
MULN R1 2  
STORE R1 n
```

```
OUTSN 1  
LOADM R1 n  
OUTR R1 0  
OUTSN 2
```

```
JUMP start
```

```
LABEL end  
STOP
```

Virtual Machine:

```
#include <iostream>  
#include <fstream>  
#include <sstream>  
#include <string>  
#include <vector>  
#include <cstdlib>  
using namespace std;  
  
unsigned int code_memory[1000];  
int data_memory[1000];  
string string_memory[200];  
int registers[8];
```

```
int n_code = 0;
int n_string = 0;
```

```
bool trace = false;
```

```
vector<string> label_names;
vector<int> label_addr;
vector<string> vnames;
```

```
const int OP_STOP = 0;
const int OP_LOADN = 1;
const int OP_LOADM = 2;
const int OP_LOADR = 3;
const int OP_ADDR = 4;
const int OP_ADDM = 5;
const int OP_ADDN = 6;
const int OP_MULR = 7;
const int OP_MULN = 8;
const int OP_MULM = 9;
const int OP_SUBN = 10;
const int OP_SUBR = 11;
const int OP_SUBM = 12;
const int OP_DIVN = 13;
const int OP_DIVR = 14;
const int OP_DIVM = 15;
const int OP_JZER = 16;
const int OP_JNEG = 17;
const int OP_JPOS = 18;
const int OP_JUMP = 19;
const int OP_READN = 20;
const int OP_OUTR = 21;
const int OP_OUTSN = 22;
const int OP_OUTSR = 23;
const int OP_STORE = 24;
```

```
bool blank_line(string line)
{
    for (int n = 0; n < (int)line.size(); n += 1)
    {
        if (line[n] != ' ' && line[n] != '\t')
        {
            return false;
        }
    }
}
```

```

    return true;
}

void error(string message)
{
    cout << "ERROR: " << message << endl;
    cin.get();
    cin.get();
    exit(1);
}

int get_variable(const string &name)
{
    for (int n = 0; n < (int)vnames.size(); n += 1)
    {
        if (vnames[n] == name)
        {
            return n;
        }
    }

    return -1;
}

int get_address(const string &name)
{
    for (int n = 0; n < (int)label_names.size(); n += 1)
    {
        if (label_names[n] == name)
        {
            return label_addr[n];
        }
    }

    return -1;
}

int get_reg_num(string s)
{
    if (s.size() < 2 || s[0] != 'R')
    {
        error("Bad register " + s);
    }
}

```

```

int r = atoi(s.substr(1).c_str());

if (r < 0 || r >= 8)
{
    error("Register out of range");
}

return r;
}

void check_data(int x)
{
    if (x < 0 || x >= 1000)
    {
        error("Data memory out of range");
    }
}

void check_string(int x)
{
    if (x < 0 || x >= n_string)
    {
        error("String memory out of range");
    }
}

void pass1_line(string line)
{
    if (blank_line(line))
    {
        return;
    }

    if (line[0] == '#')
    {
        return;
    }

    stringstream ss(line);
    string word;
    ss >> word;

    if (word == "LABEL")

```

```

{
    string name;
    ss >> name;

    if (get_address(name) != -1)
    {
        error("Duplicate label " + name);
    }

    label_names.push_back(name);
    label_addr.push_back(n_code);
}

else if (word == "DATA")
{
    string name;
    int val;
    ss >> name >> val;

    if (get_variable(name) != -1)
    {
        error("Duplicate variable " + name);
    }

    vnames.push_back(name);
    data_memory[vnames.size() - 1] = val;
}

else if (word == "STRING")
{
    if (n_string >= 200)
    {
        error("Too many strings");
    }

    n_string += 1;
}

else
{
    if (n_code >= 1000)
    {
        error("Too many instructions");
    }
}

```

```

        n_code += 1;
    }
}

void assemble_line(string line)
{
    if (blank_line(line))
    {
        return;
    }

    if (line[0] == '#')
    {
        return;
    }

    stringstream ss(line);
    string instr;
    ss >> instr;

    if (instr == "STRING")
    {
        string str;
        ss >> str;

        for (int n = 0; n < (int)str.size(); n += 1)
        {
            if (str[n] == '_')
            {
                str[n] = ' ';
            }
            if (str[n] == '~')
            {
                str[n] = '\n';
            }
        }

        static int pos = 0;
        string_memory[pos] = str;
        pos += 1;
        return;
    }
}

```

```

if (instr == "DATA" || instr == "LABEL")
{
    return;
}

int reg = 0;
int val = 0;
string word;

if (instr == "STOP")
{
    code_memory[n_code] = OP_STOP << 24;
    n_code += 1;
    return;
}

if (instr == "JUMP")
{
    ss >> word;
    val = get_address(word);

    if (val == -1)
    {
        error("unknown label " + word);
    }

    code_memory[n_code] = OP_JUMP << 24 | val;
    n_code += 1;
    return;
}

if (instr == "OUTSN")
{
    ss >> val;
    check_string(val);

    code_memory[n_code] = OP_OUTSN << 24 | val;
    n_code += 1;
    return;
}

ss >> word;
reg = get_reg_num(word);

```

```

if (instr == "READN")
{
    code_memory[n_code] = OP_READN << 24 | reg << 16;
    n_code += 1;
    return;
}

if (instr == "OUTR")
{
    code_memory[n_code] = OP_OUTR << 24 | reg << 16;
    n_code += 1;
    return;
}

if (instr == "OUTSR")
{
    code_memory[n_code] = OP_OUTSR << 24 | reg << 16;
    n_code += 1;
    return;
}

ss >> word;

if (instr == "LOADN" || instr == "ADDN" || instr == "MULN" || instr == "SUBN" || instr ==
"DIVN")
{
    val = atoi(word.c_str());
}

else if (instr == "LOADM" || instr == "STORE" || instr == "ADDM" || instr == "MULM" || instr ==
"SUBM" || instr == "DIVM")
{
    val = get_variable(word);

    if (val == -1)
    {
        error("unknown variable " + word);
    }
}

else if (instr == "LOADR" || instr == "ADDR" || instr == "MULR" || instr == "SUBR" || instr ==
"DIVR")
{
    val = get_reg_num(word);
}

```

```

}

else if (instr == "JZER" || instr == "JNEG" || instr == "JPOS")
{
    val = get_address(word);

    if (val == -1)
    {
        error("unknown label " + word);
    }
}

if (instr == "LOADN")
{
    code_memory[n_code] = OP_LOADN << 24 | reg << 16 | val;
}

else if (instr == "LOADM")
{
    code_memory[n_code] = OP_LOADM << 24 | reg << 16 | val;
}

else if (instr == "LOADR")
{
    code_memory[n_code] = OP_LOADR << 24 | reg << 16 | val;
}

else if (instr == "STORE")
{
    code_memory[n_code] = OP_STORE << 24 | reg << 16 | val;
}

else if (instr == "ADDR")
{
    code_memory[n_code] = OP_ADDR << 24 | reg << 16 | val;
}

else if (instr == "ADDN")
{
    code_memory[n_code] = OP_ADDN << 24 | reg << 16 | val;
}

else if (instr == "ADDM")
{

```

```

        code_memory[n_code] = OP_ADDM << 24 | reg << 16 | val;
    }

else if (instr == "MULR")
{
    code_memory[n_code] = OP_MULR << 24 | reg << 16 | val;
}

else if (instr == "MULN")
{
    code_memory[n_code] = OP_MULN << 24 | reg << 16 | val;
}

else if (instr == "MULM")
{
    code_memory[n_code] = OP_MULM << 24 | reg << 16 | val;
}

else if (instr == "SUBN")
{
    code_memory[n_code] = OP_SUBN << 24 | reg << 16 | val;
}

else if (instr == "SUBR")
{
    code_memory[n_code] = OP_SUBR << 24 | reg << 16 | val;
}

else if (instr == "SUBM")
{
    code_memory[n_code] = OP_SUBM << 24 | reg << 16 | val;
}

else if (instr == "DIVN")
{
    code_memory[n_code] = OP_DIVN << 24 | reg << 16 | val;
}

else if (instr == "DIVR")
{
    code_memory[n_code] = OP_DIVR << 24 | reg << 16 | val;
}

else if (instr == "DIVM")

```

```

        {
            code_memory[n_code] = OP_DIVM << 24 | reg << 16 | val;
        }

else if (instr == "JZER")
    {
        code_memory[n_code] = OP_JZER << 24 | reg << 16 | val;
    }

else if (instr == "JNEG")
    {
        code_memory[n_code] = OP_JNEG << 24 | reg << 16 | val;
    }

else if (instr == "JPOS")
    {
        code_memory[n_code] = OP_JPOS << 24 | reg << 16 | val;
    }

else
    {
        error("unknown instruction..." + instr);
    }

n_code += 1;
}

void run()
{
    int PC = 0;

    while (true)
    {
        if (PC < 0 || PC >= n_code)
        {
            error("program counter out of range");
        }

        unsigned int instr = code_memory[PC];
        PC += 1;

        int opcode = instr >> 24;
        int reg = (instr >> 16) & 255;
        int operand = instr & 65535;
    }
}

```

```

    if (trace)
    {
        cout << "PC=" << PC - 1 << " OP=" << opcode << " R=" << reg << " N=" << operand <<
endl;
    }

    switch (opcode)
    {
    case OP_STOP:
        return;

    case OP_LOADN:
        registers[reg] = operand;
        break;

    case OP_LOADM:
        check_data(operand);
        registers[reg] = data_memory[operand];
        break;

    case OP_LOADR:
        registers[reg] = registers[operand];
        break;

    case OP_STORE:
        check_data(operand);
        data_memory[operand] = registers[reg];
        break;

    case OP_ADDR:
        registers[reg] += registers[operand];
        break;

    case OP_ADDM:
        check_data(operand);
        registers[reg] += data_memory[operand];
        break;

    case OP_ADDN:
        registers[reg] += operand;
        break;

    case OP_MULR:

```

```
    registers[reg] *= registers[operand];  
    break;
```

```
case OP_MULM:  
    check_data(operand);  
    registers[reg] *= data_memory[operand];  
    break;
```

```
case OP_MULN:  
    registers[reg] *= operand;  
    break;
```

```
case OP_SUBR:  
    registers[reg] -= registers[operand];  
    break;
```

```
case OP_SUBM:  
    check_data(operand);  
    registers[reg] -= data_memory[operand];  
    break;
```

```
case OP_SUBN:  
    registers[reg] -= operand;  
    break;
```

```
case OP_DIVR:  
    if (registers[operand] == 0)  
        error("divide by zero");  
    registers[reg] /= registers[operand];  
    break;
```

```
case OP_DIVM:  
    check_data(operand);  
    if (data_memory[operand] == 0)  
        error("divide by zero");  
    registers[reg] /= data_memory[operand];  
    break;
```

```
case OP_DIVN:  
    if (operand == 0)  
        error("divide by zero");  
    registers[reg] /= operand;  
    break;
```

```

case OP_JZER:
    if (registers[reg] == 0)
        PC = operand;
    break;

case OP_JNEG:
    if (registers[reg] < 0)
        PC = operand;
    break;

case OP_JPOS:
    if (registers[reg] > 0)
        PC = operand;
    break;

case OP_JUMP:
    PC = operand;
    break;

case OP_READN:
    cin >> registers[reg];
    break;

case OP_OUTR:
    cout << registers[reg];
    cout.flush();
    break;

case OP_OUTSN:
    check_string(operand);
    cout << string_memory[operand];
    cout.flush();
    break;

case OP_OUTSR:
    check_string(registers[reg]);
    cout << string_memory[registers[reg]];
    cout.flush();
    break;

default:
    error("bad opcode");
}
}

```

```
}
```

```
int main(int argc, char *argv[])
```

```
{
```

```
    if (argc < 2)
```

```
    {
```

```
        cout << "Please provide an assembly file XD" << endl;
```

```
        cout << "For example: assembler program.asm" << endl;
```

```
        return 1;
```

```
    }
```

```
    ifstream fin("program.asm");
```

```
    if (!fin)
```

```
    {
```

```
        cout << "Sorry can't open file :(" << endl;
```

```
        return 1;
```

```
    }
```

```
    string line;
```

```
    while (getline(fin, line))
```

```
    {
```

```
        pass1_line(line);
```

```
    }
```

```
    fin.clear();
```

```
    fin.seekg(0);
```

```
    n_code = 0;
```

```
    while (getline(fin, line))
```

```
    {
```

```
        assemble_line(line);
```

```
    }
```

```
    run();
```

```
    return 0;
```

```
}
```

Results:

Enter a number: 55
The double is: 110
Enter a number: 110
The double is: 220
Enter a number: 220
The double is: 440
Enter a number: 440
The double is: 880
Enter a number: 880
The double is: 1760
Enter a number: 1760
The double is: 3520
Enter a number: 3520
The double is: 7040
Enter a number: 7040
The double is: 14080
Enter a number: 14080
The double is: 28160
Enter a number: 28160
The double is: 56320
Enter a number: 56320
The double is: 112640
Enter a number: 112640
The double is: 225280
Enter a number: 225280
The double is: 450560
Enter a number: 450560
The double is: 901120
Enter a number: 901120
The double is: 1802240
Enter a number: 1802240
The double is: 3604480
Enter a number: 3604480
The double is: 7208960
Enter a number: 7208960
The double is: 14417920
Enter a number: 14417920
The double is: 28835840

Enter a number: 28835840
The double is: 57671680
Enter a number: 57671680
The double is: 115343360
Enter a number: 115343360
The double is: 230686720
Enter a number: 230686720
The double is: 461373440
Enter a number: 461373440
The double is: 922746880
Enter a number: 92246880
The double is: 184493760
Enter a number: 184493760
The double is: 368987520
Enter a number: 368987520
The double is: 737975040
Enter a number: 737975040
The double is: 1475950080